

Neural Population Inference for Gravitational Wave Astrophysics via Conditional Flow Matching

Overview

Gravitational wave detections from LIGO and Virgo have revealed populations of merging black hole binaries, but the astrophysical processes that form these systems remain poorly understood. Determining the formation physics, such as common envelope efficiency (α_{CE}) and natal black hole spins (χ_b), requires comparing predicted merger populations to observed catalogs. Current methods rely on Markov Chain Monte Carlo sampling through computationally expensive population synthesis codes like COMPAS, limiting exploration of the high-dimensional hyperparameter space.

Our project develops an amortized neural inference framework that enables rapid Bayesian inference on astrophysical hyperparameters from gravitational wave catalogs. We train conditional generative models (Conditional Flow Matching networks) to emulate COMPAS population synthesis at scale, then build inference networks that directly map observed catalogs to posterior distributions over formation-physics parameters. This approach aims to transform population inference from weeks of MCMC sampling to minutes of neural network evaluation, enabling systematic uncertainty quantification and comparison across competing formation channels.

The trained models will be made publicly available, allowing the gravitational wave community to perform population-level inference without running expensive COMPAS simulations. This work bridges machine learning and gravitational wave astrophysics, introducing flow matching, a state-of-the-art generative modeling technique, to the domain for the first time.

Computational Requirements

Phase 1: Population Synthesis Data Generation

To train high-fidelity neural emulators, we require comprehensive training and validation datasets covering a wide range of astrophysical hyperparameters. We will generate these datasets using SSPC (<https://synthetic-stellar-pop-convolve.readthedocs.io>), a GPU-accelerated framework that interfaces with COMPAS for efficient large-scale population synthesis.

Simulation Plan:

We will generate 10,000–100,000 independent population synthesis runs spanning a Latin hypercube sample across a 6-10 dimensional hyperparameter space.

Computational Scale:

- **Each SSPC run:** $\sim 10^4$ binaries simulated
- **CPU Time per Binary:** ~ 1 second
- **Memory:** ~ 4 GB per node
- **For 10,000 runs:**
 - 10^4 runs $\times 10^4$ binaries $\times 1$ sec = 30,000 CPU-hours
- **Timeline:** 2-3 weeks on Expanse CPU nodes (assuming 500-1000 core allocation)

Phase 2: Forward Model Training (Primary GPU Need)

We train a Conditional Flow Matching (CFM) emulator that learns to generate synthetic gravitational wave merger catalogs conditioned on astrophysical hyperparameters. The CFM is a neural ordinary differential equation (ODE) model with $\sim 800,000$ parameters that learns the joint distribution of four observables (chirp mass, mass ratio, effective spin, redshift) across five formation channels (common envelope, chemically homogeneous evolution, globular cluster, nuclear star cluster, stable mass transfer).

Training dataset: The 10,000 SSPC simulations from Phase 1 containing $\sim 10^6$ - 10^7 total merger events (COMPAS runs) per simulation across hyperparameter space.

GPU Requirements:

- **Training:** 100,000 gradient steps with batch size 512 (mixed precision FP16)
- **Model architecture:**
 - Event encoder: $7 \rightarrow 256 \rightarrow 256 \rightarrow 256 \rightarrow 256$ (encoding hyperparameters)
 - Vector field network: $133 \rightarrow 512 \rightarrow 512 \rightarrow 512 \rightarrow 512 \rightarrow 4$ (learning flow dynamics)
- **ODE solver:** Adaptive Runge-Kutta (dopri5) for sampling, requires ~ 50 function evaluations per event
- **Memory:** ~ 12 GB GPU memory per training run
- **Time per run:** ~ 8 -12 hours on NVIDIA V100
- **Production Runs:**
 - 1 joint CFM model (all channels): 12 GPU-hours
 - 5 per-channel CFM models (contingency): $5 \times 10 = 50$ GPU-hours
 - 5 bootstrap ensemble models (uncertainty quantification): $5 \times 12 = 60$ GPU-hours
 - Hyperparameter tuning (learning rate, architecture variants): 20 GPU-hours
 - **Subtotal production runs:** ~ 142 GPU-hours
- **Test/Development Runs:**
 - Data pipeline validation (small scale runs): 10 GPU-hours
 - Architecture debugging (smoke tests, gradient checks): 15 GPU-hours
 - Hyperparameter exploration (learning rate schedules, batch sizes): 21 GPU-hours
 - Failed runs and reruns: ~ 20 GPU-hours
 - Validation experiments (ablation studies): 5 GPU-hours
 - **Subtotal test/development:** ~ 71 GPU-hours
- **Total phase:** ~ 213 GPU-hours

Phase 3: Inference Network Training

We train a Transformer-based inference network (200,000 parameters) that processes variable-size gravitational wave catalogs (50-100 events) and outputs posterior distributions over astrophysical hyperparameters. The transformer encoder aggregates information from noisy per-event measurements, and a normalizing flow decoder produces the posterior.

GPU Requirements:

- **Training:** 50,000 gradient steps, batch size 64 catalogs
- **Synthetic data generation:** Use trained CFM to generate 50,000 synthetic catalogs (5 GPU-hours)
- **Time per run:** ~6 hours on V100
- **Production Runs:**
 - 1 inference network: 6 GPU-hours
 - 5 ensemble members: $5 \times 6 = 30$ GPU-hours
 - Architecture search and validation: 10 GPU-hours
 - **Subtotal production runs:** ~46 GPU-hours for inference network training
- **Test/Development Runs:**
 - Synthetic catalog generation experiments: 5 GPU-hours
 - Transformer architecture tuning: 8 GPU-hours
 - Training stability tests: 5 GPU-hours
 - Failed runs and reruns: 5 GPU-hours
 - **Subtotal test/development:** 23 GPU-hours
- **Total phase: 69 GPU-hours**

Phase 4: Validation & Real Data Analysis

- **Production Runs:**
 - Simulation-based calibration (generating validation sets): 10 GPU-hours
 - Final model evaluation on test sets: 3 GPU-hours
 - GWTC-3 catalog analysis (~90 events): CPU inference (<1 minute), 2 GPU-hours
 - **Subtotal production:** 15 GPU-hours
- **Test/Development Runs:**
 - Validation pipeline debugging: 4 GPU-hours
 - Calibration experiments: 4 GPU-hours
 - **Subtotal test/development:** 8 GPU-hours
- **Total phase: 23 GPU-hours**

Total Resource Summary

Phase	CPU-hours	GPU-hours (Production)	GPU-hours (Test/Dev)	GPU-hours (Total)
1: SSPC/COMPAS	30,000	~	~	~
2: CFM Training	~	142	71	213
3: Inference Training	~	46	23	69
4: Validation	~	15	8	23
TOTAL	30,000	203	102	305

Storage Requirements:

- Raw SSPC outputs: ~2 TB (compressed HDF5 files)
- Processed ML datasets: ~200 GB
- Training checkpoints: ~300 GB (intermediate, deleted after validation)
- Final trained models (for public release): ~5 GB
- Analysis outputs: ~100 GB
- **Total storage: ~2.6 TB**

Neural network training is inherently iterative and requires extensive experimentation. We build in a buffer to account for:

- Data pipeline validation and debugging
- Hyperparameter tuning (learning rates, batch sizes, architecture choices)
- Initial runs that fail due to numerical instabilities or configuration errors
- Convergence validation and potential restarts with adjusted settings
- Ablation studies to understand architectural contributions

This conservative buffer ensures project completion within the allocated timeframe without requiring supplemental requests.

Why ACCESS Expanse?

Expanse is well-suited to this project because:

1. **Hybrid CPU+GPU Architecture:** Our workflow requires both CPU-intensive population synthesis (Phase 1) and GPU-accelerated neural network training (Phases 2-3). Expanse's integrated CPU and GPU partitions enable seamless workflow execution on a single system without cross-machine data transfers.

2. **GPU Performance for Neural ODEs:** Expanse provides NVIDIA V100 (32GB) GPUs, which are ideal for training neural ODEs with mixed-precision. Our models fit comfortably in 8-16 GB memory and benefit from tensor core acceleration.
3. **Fast Turnaround:** Our training runs are short (4-12 hours), allowing rapid iteration during model development and hyperparameter tuning. Expanse's queue structure accommodates this workflow.
4. **Local to UCSD:** The PI is based at UC San Diego. The PI holds an active SDSC account and has prior experience with SLURM job submission on HPC systems.
5. **Software Stack:** Expanse supports PyTorch, torchdiffeq (ODE solver), and standard ML frameworks out-of-the-box, minimizing setup time.